به نام خدا

دانشگاه تهران
دانشکدگان فنی
دانشکده مهندسی برق و کامپیوتر

درس مدلسازی و درستی‌یابی صوری

گزارش تمرین برنامه‌نویسی ۱

نام و نام خانوادگی :
فاطمه عبدلی
هانیه گرگانی

شماره دانشجویی:
۸۱۰۱۰۴۲۰۲
۸۱۰۱۰۴۲۳۸

آبان ۱۴۰۴

1. Introduction and goals

The First phase of this project established a basic communication model for three satellites and a ground station, employing a Time Division Multiple Access (TDMA) protocol with 8 slots (0-7) to manage inter-satellite (ISL) and satellite-to-ground communications to avoid conflict and congestion. Also message buffering was implemented due to asynchronous message passing. This phase enhances the model's functionalities and reliability through two major additions which are Energy Management and Formal Verification.

1.1 Energy Management

This part developed in order to simulate realistic power constraints and system behavior under low energy to prevent mission-critical failure and ensure autonomous recovery.

1.2 Formal Verification

LTL properties and assertions prove system correctness. Checks for Safety- no ground conflict, no low-energy send, and no buffer overflow- and Liveness- guaranteed message delivery/drop, and periodic ground access- properties are implemented.

2. Structure and code

The model is written in ***Promela*** (Process Meta Language) for the ***Spin*** Model Checker, and uses channels for communication and asynchronous message passing.

2.1 Constant values

This section defines the core parameters, particularly the energy management thresholds and costs that govern the satellite behavior. Constants shown in Figure1.1 clearly establish the power budget. Sending to the Ground (15

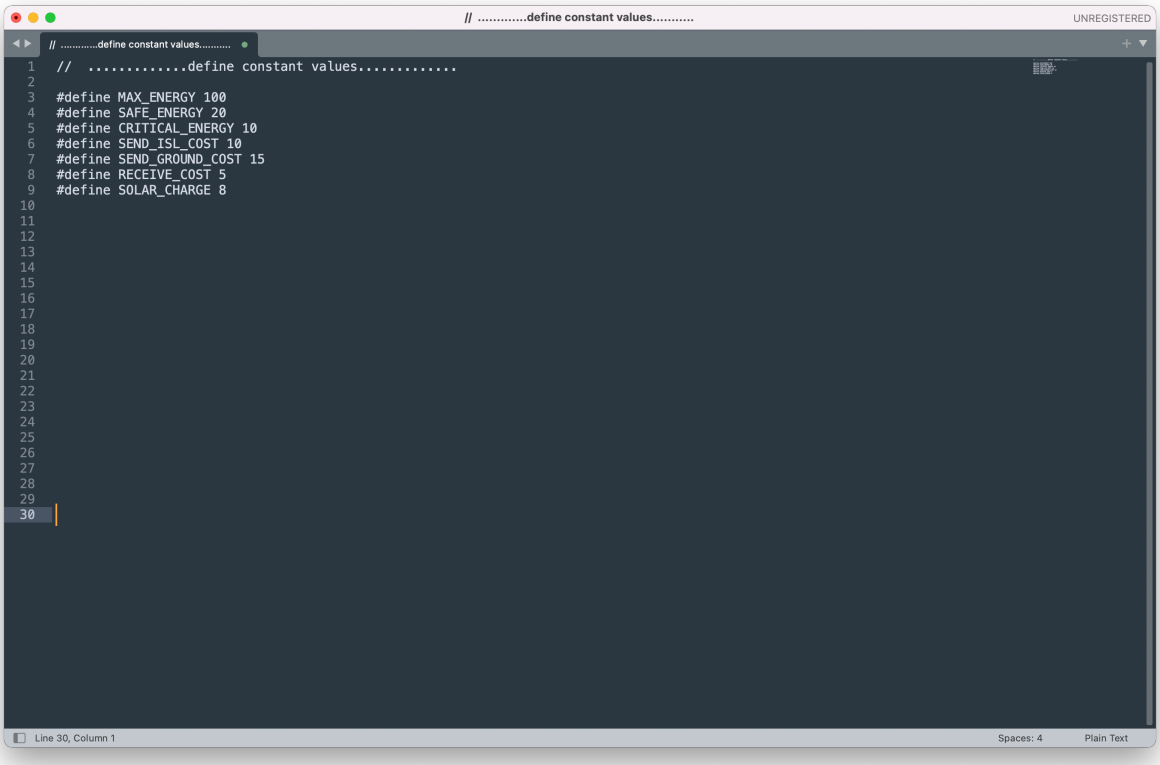


Figure1.1- definition of constant values

units) is the most expensive operation, promoting a conservative approach, while Solar charge (8 units) is the recharging amount. The Safe (20) and Critical (10) bounds dictate the satellite's ability to operate.

2.2 Timekeeper and Energy Recharge

The timekeeper process guarantees the system's **Liveness** by ensuring energy levels trend upwards. Critically, as shown in Figure1.2 it checks the Safe Mode exit condition and resets the safe_mode flag, allowing the satellite to resume transmitting.

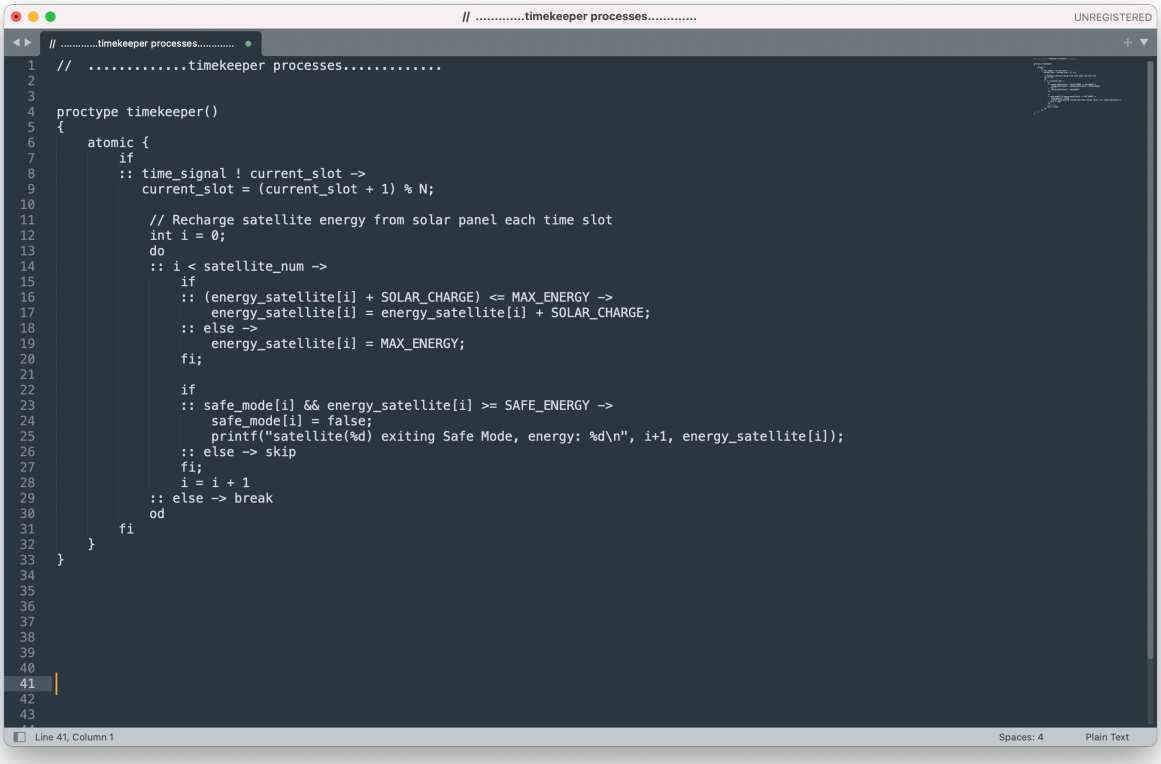


Figure1.2- definition of timekeeper process

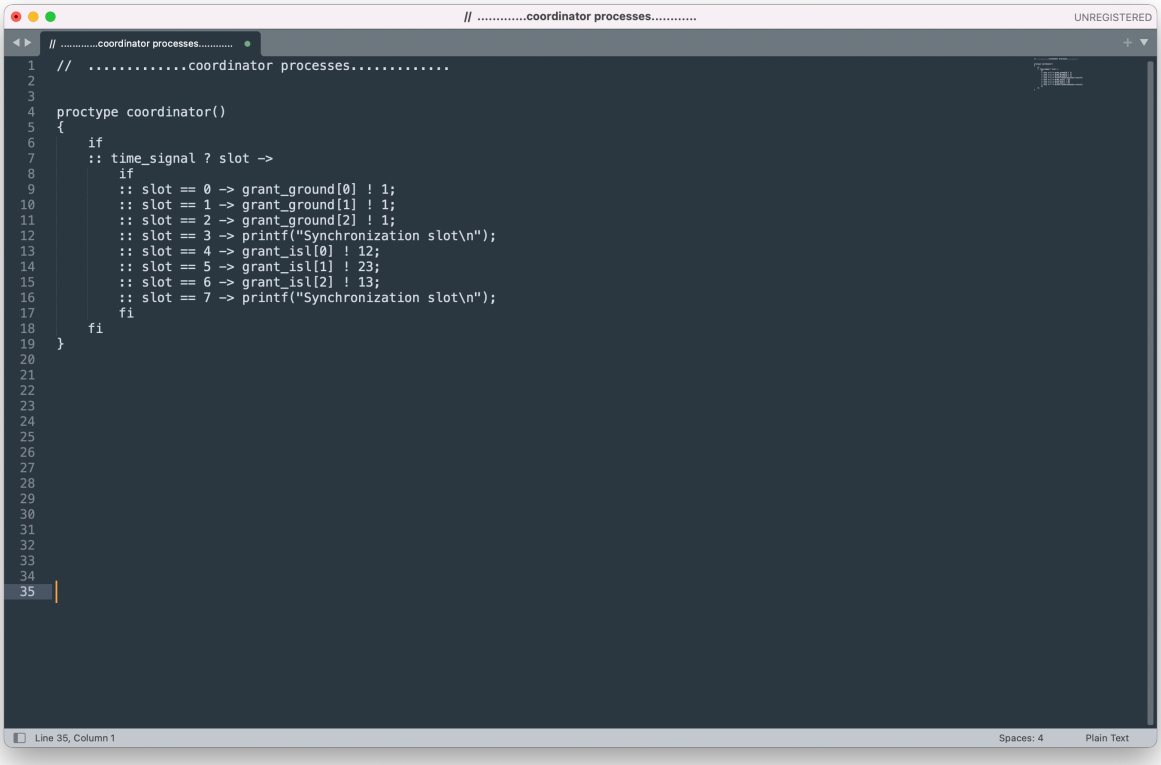


Figure1.3- definition of coordinator process

2.3 TDMA Coordinator

The coordinator process shown in Figure1.3 manages the 8-slot TDMA schedule and grants access to the shared communication channels. In this phase we've fixed schedule to ensures **Liveness** periodic access property. Satellites 1, 2, and 3 get dedicated ground slots 0, 1, 2 and dedicated ISL slots 4, 5, 6. Slots 3 and 7 are reserved for synchronization as before.

2.4 Satellite

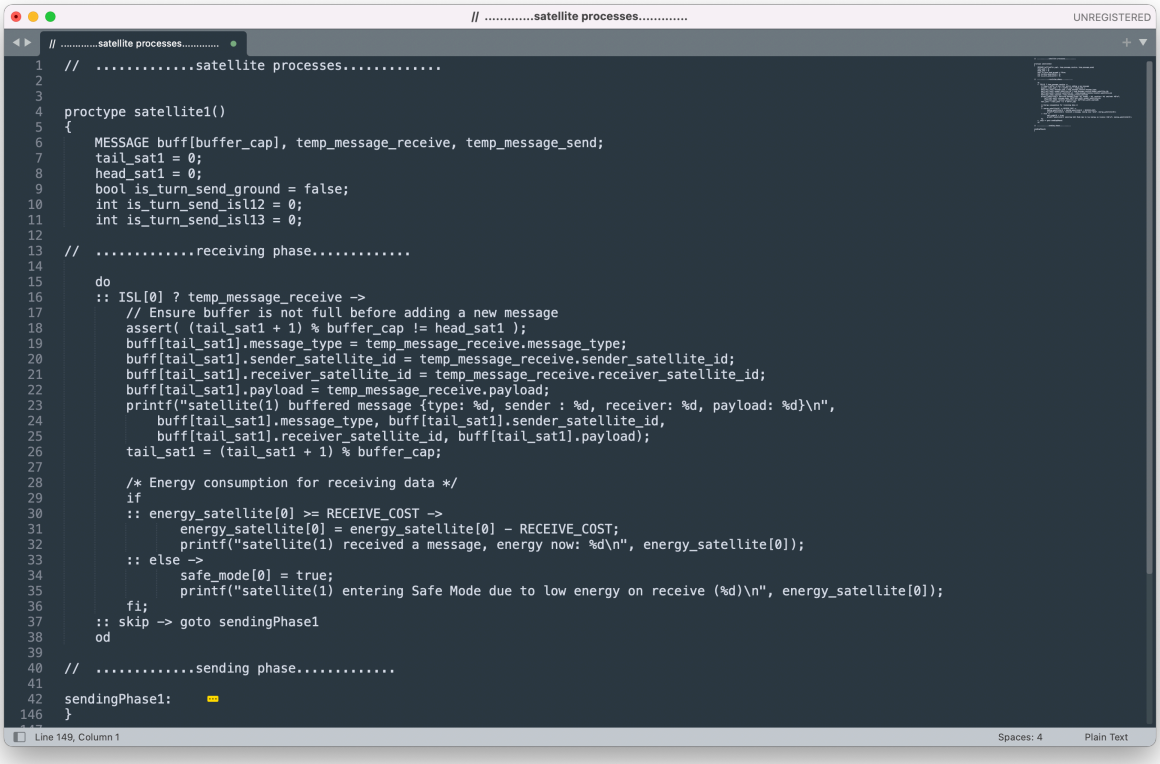
2.4.1 Receiving phase

In this section two key safety rules are enforced, shown in Figure1.4:

- **No Buffer Overflow:** The assert(...) explicitly checks this **safety property**.
- **Safe Mode Entry:** If the satellite cannot afford the RECEIVE_COST which is 5 units, or if the cost pushes it below the **Critical Energy Bound**, it enters safe_mode.

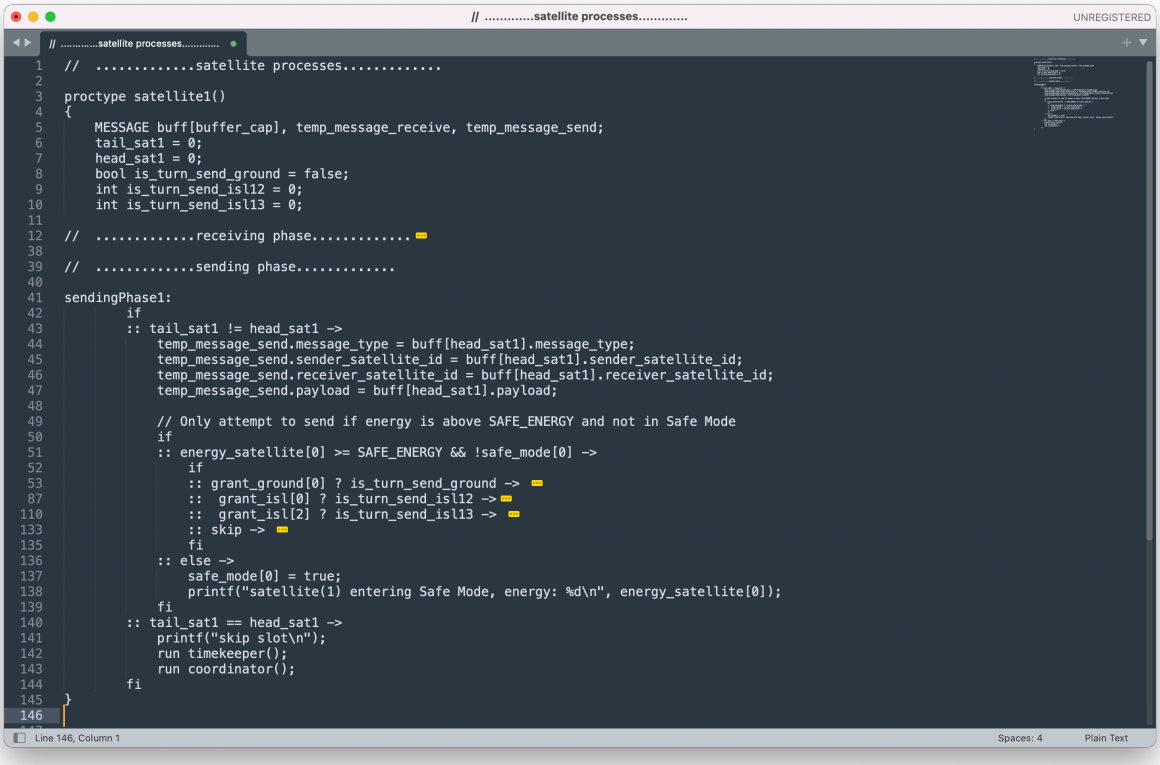
2.4.2 Sending Phase

Here **TDMA protocol** and **Energy Management** rules are strictly enforced before a message can be transmitted. The process first checks if there is a message ready to be sent, and if is, it checks the satellite's energy level to be upper both safe and critical bounds. And if not it will skip the slot as shown in Figure 1.5.



```
1   //  .............satellite processes.............
2
3
4   proctype satellite1()
5   {
6       MESSAGE buff[buffer_cap], temp_message_receive, temp_message_send;
7       tail_sat1 = 0;
8       head_sat1 = 0;
9       bool is_turn_send_ground = false;
10      int is_turn_send_isl12 = 0;
11      int is_turn_send_isl13 = 0;
12
13  //  .............receiving phase.............
14
15      do
16      :: ISL[0] ? temp_message_receive ->
17          // Ensure buffer is not full before adding a new message
18          assert( (tail_sat1 + 1) % buffer_cap != head_sat1 );
19          buff[tail_sat1].message_type = temp_message_receive.message_type;
20          buff[tail_sat1].sender_satellite_id = temp_message_receive.sender_satellite_id;
21          buff[tail_sat1].receiver_satellite_id = temp_message_receive.receiver_satellite_id;
22          buff[tail_sat1].payload = temp_message_receive.payload;
23          printf("satellite(1) buffered message {type: %d, sender : %d, receiver: %d, payload: %d}\n",
24              buff[tail_sat1].message_type, buff[tail_sat1].sender_satellite_id,
25              buff[tail_sat1].receiver_satellite_id, buff[tail_sat1].payload);
26          tail_sat1 = (tail_sat1 + 1) % buffer_cap;
27
28          /* Energy consumption for receiving data */
29          if
30          :: energy_satellite[0] >= RECEIVE_COST ->
31                energy_satellite[0] = energy_satellite[0] - RECEIVE_COST;
32                printf("satellite(1) received a message, energy now: %d\n", energy_satellite[0]);
33          :: else ->
34                safe_mode[0] = true;
35                printf("satellite(1) entering Safe Mode due to low energy on receive (%d)\n", energy_satellite[0]);
36          fi;
37      :: skip -> goto sendingPhase1
38      od
39
40  //  .............sending phase.............
41
42  sendingPhase1:    ...
146 }
```

Figure1.4- definition of satellite process (receiving phase)



```
1   //  .............satellite processes.............
2
3   proctype satellite1()
4   {
5       MESSAGE buff[buffer_cap], temp_message_receive, temp_message_send;
6       tail_sat1 = 0;
7       head_sat1 = 0;
8       bool is_turn_send_ground = false;
9       int is_turn_send_isl12 = 0;
10      int is_turn_send_isl13 = 0;
11
12  //  .............receiving phase.............  ...
38
39  //  .............sending phase.............
40
41  sendingPhase1:
42          if
43          :: tail_sat1 != head_sat1 ->
44              temp_message_send.message_type = buff[head_sat1].message_type;
45              temp_message_send.sender_satellite_id = buff[head_sat1].sender_satellite_id;
46              temp_message_send.receiver_satellite_id = buff[head_sat1].receiver_satellite_id;
47              temp_message_send.payload = buff[head_sat1].payload;
48
49              // Only attempt to send if energy is above SAFE_ENERGY and not in Safe Mode
50              if
51              :: energy_satellite[0] >= SAFE_ENERGY && !safe_mode[0] ->
52                  if
53                  :: grant_ground[0] ? is_turn_send_ground ->  ...
87                  ::  grant_isl[0] ? is_turn_send_isl12 -> ...
110                 ::  grant_isl[2] ? is_turn_send_isl13 ->  ...
133                 :: skip ->  ...
135                 fi
136             :: else ->
137                 safe_mode[0] = true;
138                 printf("satellite(1) entering Safe Mode, energy: %d\n", energy_satellite[0]);
139             fi
140         :: tail_sat1 == head_sat1 ->
141             printf("skip slot\n");
142             run timekeeper();
143             run coordinator();
144         fi
145 }
146
```

Figure1.5- definition of satellite process (sending phase)

2.4.2.1 Send to the Ground

As shown in Figure1.6, if the satellite can send to the ground due to its turn and there is a message to be sent, the assert (energy_satellite[0] >= SAFE_ENERGY) reinforces no low-energy **safety property,** and also the assert (len(message_sent_to_ground) <= 1) prevents Ground conflict **safety property.** Afterward, a high-cost ground transmission is processed, and if the remainder is low, the satellite enters Safe Mode.

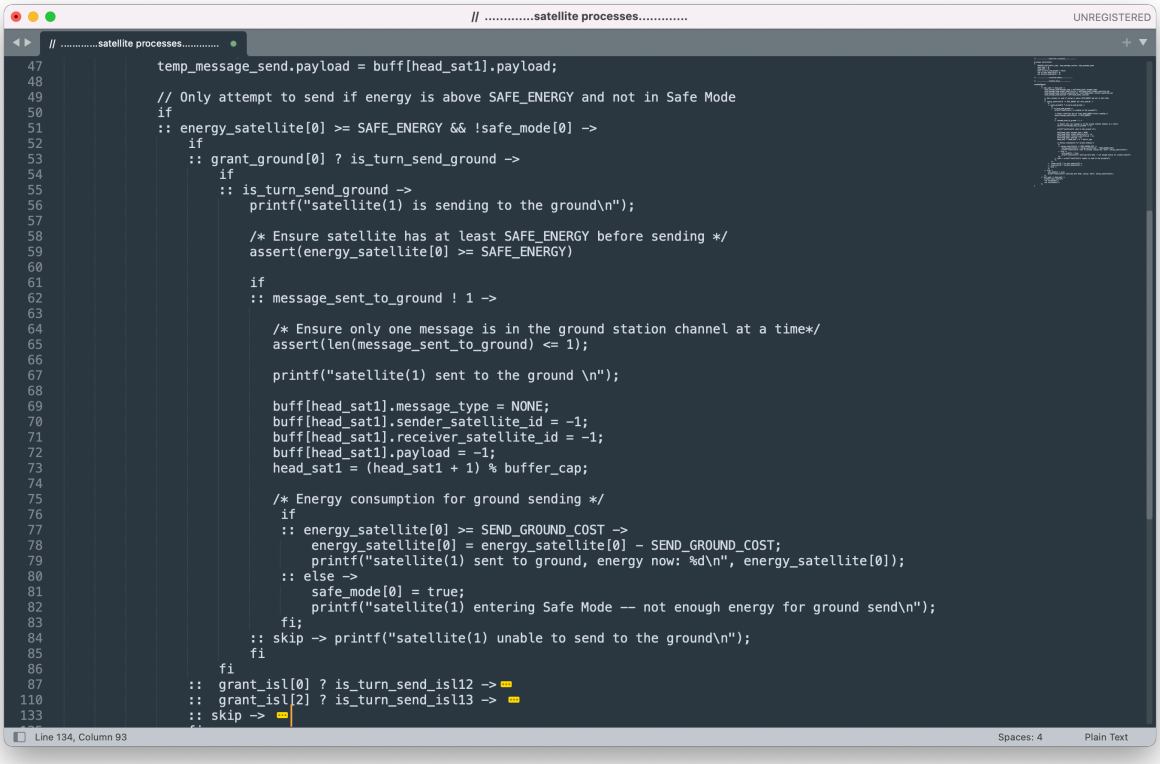


Figure1.6- definition of satellite process (sending phase - Send to the Ground)

2.4.2.2 Send to other satellites

At the very beginning of this part, the no low-energy **safety property is** checked by assert(energy_satellite[i] >= SAFE_ENERGY) and then the consumption logic is applied, ensuring all transmission activities are energy-aware and guarded against low-power operation. Each satellite (i) can send to two other satellites (j) and (k) in the orbit and the sending parts are logically the same but divided by two guards in the If Statement, shown in Figure1.7 and Figure1.8.

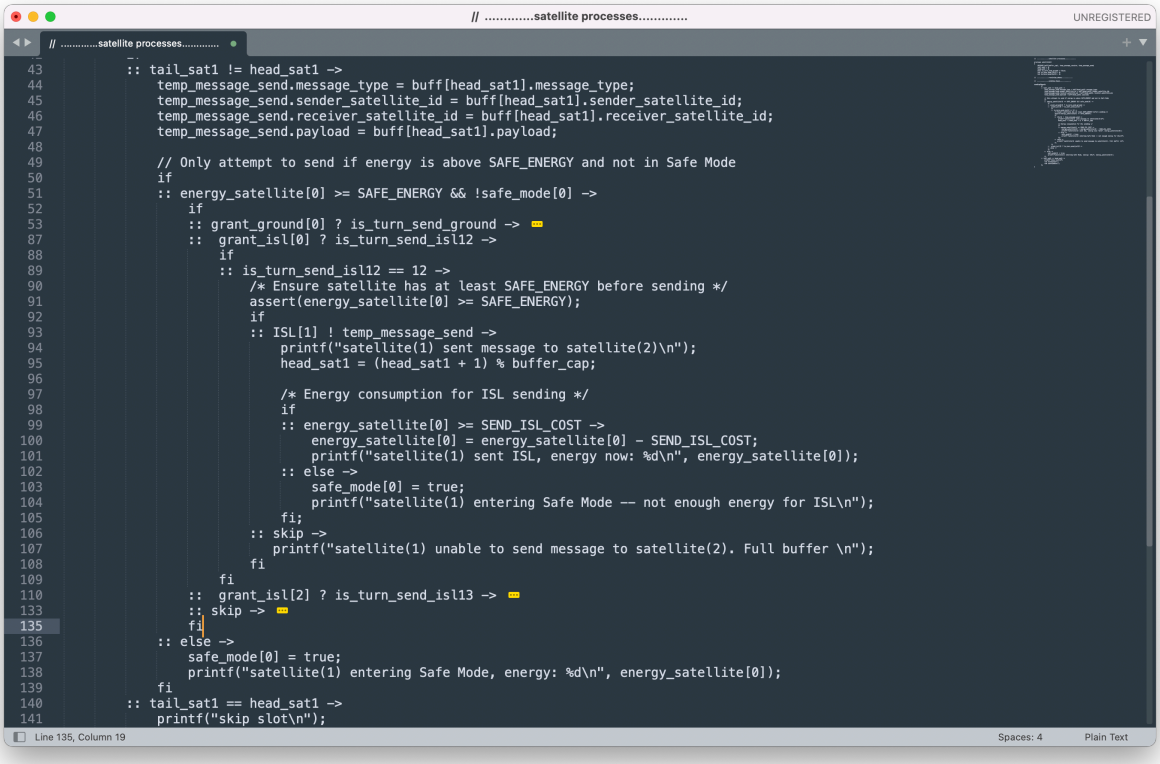


Figure1.7- definition of satellite process (sending phase - Satellite (i) send to the satellite (j))

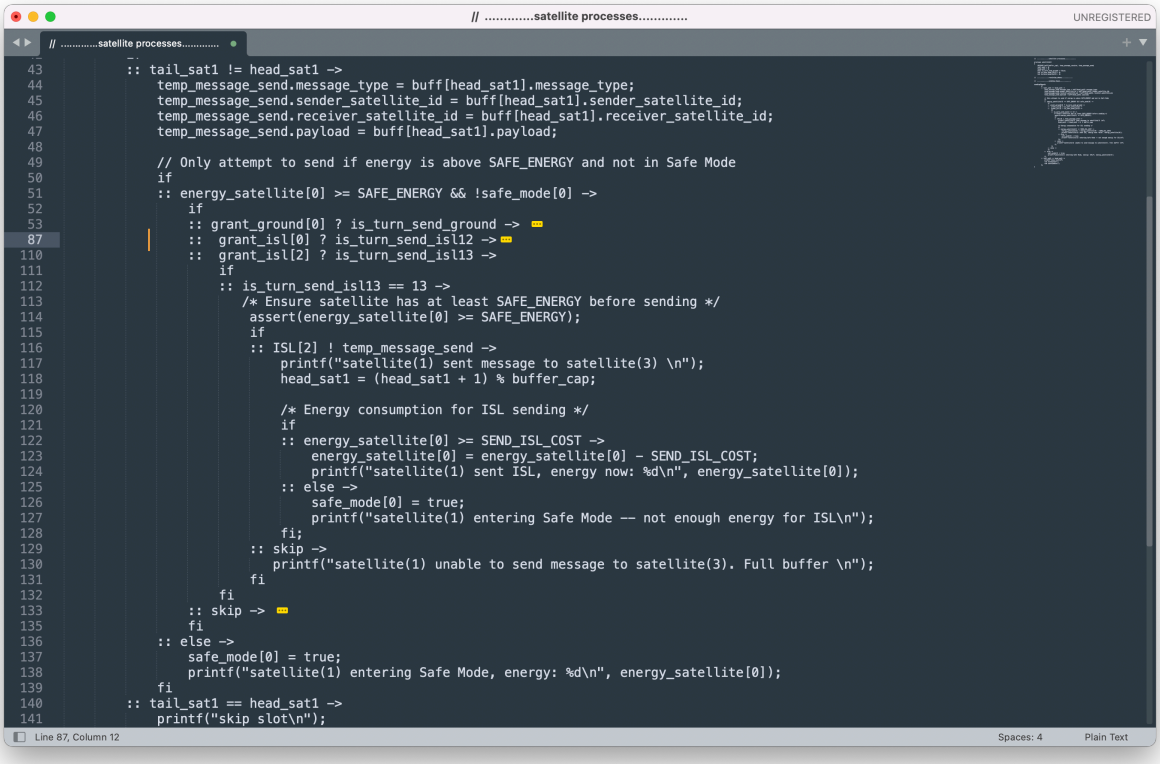


Figure1.8- definition of satellite process (sending phase - Satellite (i) send to the satellite (k))

2.5 Ground Processes

The groundStation process remained the same as what was in the Version1 as shown in Figure1.9; receives messages and confirms successful transmission and records the number of messages per satellite.

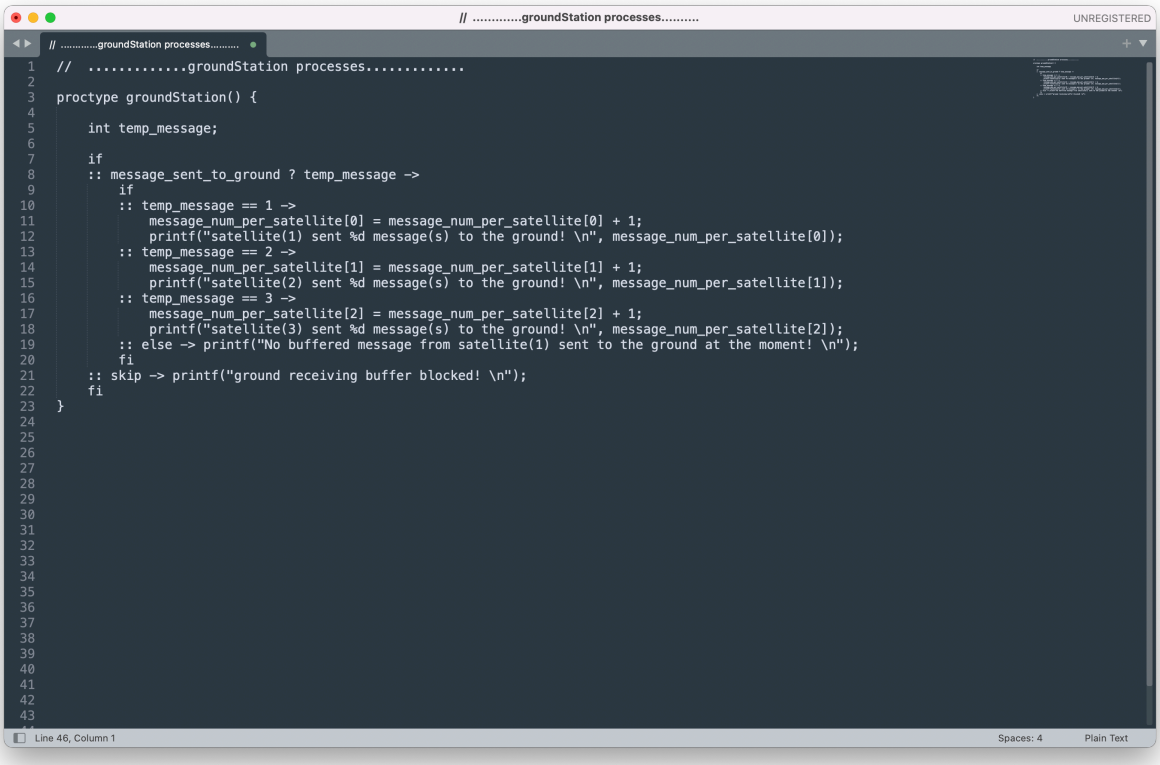


Figure1.9- definition of groundStation process

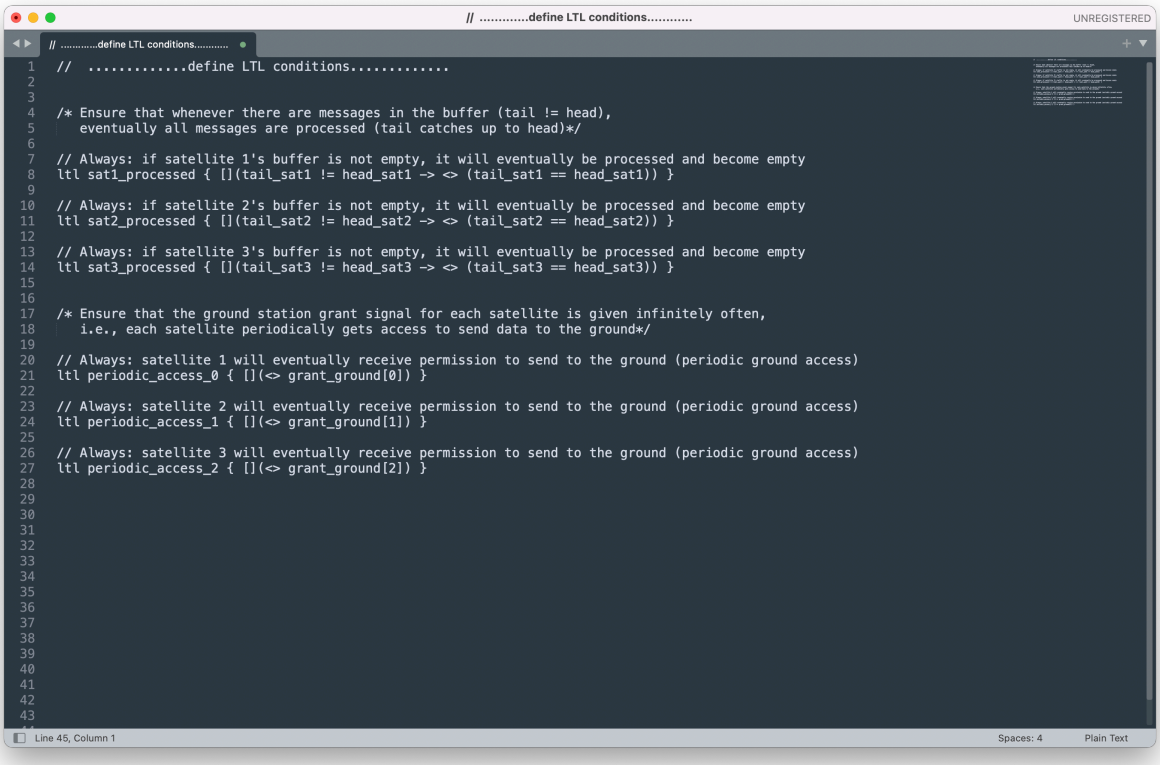


Figure1.10- definition of LTL properties

2.6 LTL Properties

Two series of LTL properties are defined as shown in Figure1.10: **Guaranteed Message Processing** and **Periodic Access**. These properties are crucial for formal verification, ensuring the satellite system does not suffer from permanent stalls or starvation.

2.6.1 Guaranteed Message Processing

These LTL properties ensures that messages placed in a satellite's buffer are not kept indefinitely and no data starvation happens; they must *eventually* be sent or dropped. If Satellite (i)'s buffer is **not empty**, it must **eventually** become **empty.**

2.6.2 Periodic Ground Access

These This LTL properties ensures that the TDMA scheduling mechanism provides every satellite with an opportunity to communicate with the ground station infinitely often, since it is cyclical. It is **eventually** true that Satellite (i) receives the **ground grant** channel.

3. How to run

3.1 Prerequisites

Spin Model Checker: The core tool required to run *Promela* code.

3.2 Execution

To see the flow of messages, energy levels, and Safe Mode entries/exits, clone repository:

Git clone https://github.com/HaniehGRN/inter-satellite-communication

And run a non-deterministic simulation in *jspin*!

4. Analysis and Results

The model correctly enforces the defined safety requirements which guarantee that specific, undesirable states are unreachable:

4.1 Safety Properties

4.1.1 No Conflict in Ground Channel

- **Assertion:** assert(len(message_sent_to_ground) <= 1);
- **Result:** This is checked immediately after a satellite attempts to send to the ground, ensuring that no more than one message ever enters the ground channel at a time.

4.1.2 No Sending Below Safe Energy

- **Assertion:** assert(energy_satellite[i] >= SAFE_ENERGY);
- **Result:** This check is executed before *a* send operation confirming the **Guard Logic** (energy_satellite[i] >= SAFE_ENERGY && !safe_mode[i]) is effective.

4.1.3 No Buffer Overflow

- **Assertion:** assert((tail_sat(i) + 1) % buffer_cap != head_sat(i));
- **Result:** This circular buffer assertion prevents data loss due to attempting to write to an already full buffer.

4.2 Liveness Properties

4.2.1 Guaranteed Message Delivery/Drop

- **LTL**: [](tail_sat(i) != head_sat(i) -> <> (tail_sat(i) == head_sat(i)))
- **Result**: The TDMA slot guarantees sending opportunities, and the Safe Mode recovery ensures that even if a satellite runs out of energy, it will eventually recharge, exit Safe Mode, and process its buffer. This guarantees the buffer will not hold data indefinitely.

4.2.2 Periodic Ground Communication

- **LTL**: [](<> grant_ground[i])
- **Result**: The cyclical TDMA schedule - slots 0, 1, and 2 are dedicated to the three satellites - ensures that every satellite receives the ground grant infinitely often.

4.3 Energy Management

state transitions are modeled based on battery level, which has three stages: Normal Operation, between MAX_ENERGY and SAFE_ENERGY. Safe Mode, lower than SAFE_ENERGY and CRITICAL_ENERGY. And Critical lower than CRITICAL_ENERGY. Each satellite gains energy at each slot, therefore, it can operate correctly.